Crystal Reports in SRamil

How the Crystal reports work and how to build a new one, from design to the button on the page · for Luís

> Crystal Reports is SAP's classic report designer: an **.rpt file** holds the layout and the data binding. SRamil does **not** run the Crystal runtime — the .rpt files live on the AgileInCloud server and are rendered by a shared service (the same one GivingCare uses). The app just asks *"render this report against this database"* and receives the finished PDF.

1. The flow, end to end

1. The user clicks a report button on a page — the shared component `GerarRelatorioCrystalButton` (in `Shared/Components`).
2. The component reads the .rpt path from `appsettings.json` and, when the report is record-filtered, the selection formula — replacing the placeholder (e.g. `{equipment_id}`) with the real Id.
3. `CrystalReportApiService` (`Services/CrystalReports`) sends a SOAP `GeneratePDF` request to `https://support.agileincloud.com/Online/AgilePDFgenerator.asmx` with: the service key (WsKey), the .rpt path, the selection formula, the SQL server, credentials and the company database name.
4. The remote service opens the .rpt, binds it to the given database, applies the formula and returns the file as Base64. The app decodes it and hands the download to the browser.

> Important: the report is executed by the **AgileInCloud server**. The .rpt must be there (not on your PC), and the database must be reachable from there.

2. Configuration (appsettings.json)

Everything lives under `CrystalReportsService`. Each new report adds a path key and, if record-filtered, a formula key. SQL credentials are not configured here — the component reuses `Client:Database`, and the database is always the session company's (multi-tenant for free).

```
"CrystalReportsService": {
  "BaseUrl":  "https://support.agileincloud.com/Online/AgilePDFgenerator.asmx",
  "WsKey":    "(service key — do not share)",
  "EquipamentosReportPath":        "C:\\AGILE_REPORTS\\SRAMIL\\Relatorio_equipamentos.rpt",
  "EquipamentosSelectionFormula":  "{Command.Id} = {equipment_id}",
  "ManutencaoReportPath":          "C:\\AGILE_REPORTS\\SRAMIL\\...rpt"
}
```

3. Designing a report in the Crystal Reports Designer

1. Open SAP Crystal Reports and create a **Blank Report**.
2. In **Database Expert**, create an OLE DB (ADO) connection → "Microsoft OLE DB Driver for SQL Server" → server, user, database (use the test one; in production the service swaps the connection for the right database).
3. Prefer a **Command** (a SQL query) over dragging tables: write the SELECT with the joins and final column names. Fields appear as `{Command.ColumnName}` — which is why selection formulas use `{Command.Id}`.
4. Layout by sections: **Report Header** (logo + title), **Page Header** (column headers), **Details** (one row per record), **Report Footer** (totals), **Page Footer** (page numbers). Drag fields from the Field Explorer into Details.
5. Totals: right-click a field → **Insert Summary** (Sum/Count) into the Report Footer.
6. Save **without data**: File → uncheck "Save Data with Report" (otherwise the .rpt ships with a snapshot of the test database embedded).

4. Selection formulas and placeholders

The selection formula is Crystal's WHERE — its own syntax, fields in braces. In SRamil the dynamic value arrives by text substitution: the appsettings key holds a token and the button component swaps it for the real value before sending.

```
// appsettings.json
"EquipamentosSelectionFormula": "{Command.Id} = {equipment_id}"

// On the page, the button declares the token and the value:
PlaceholderToken="{equipment_id}"  PlaceholderValue="@EquipmentId.ToString()"

// What reaches Crystal:  {Command.Id} = 42
```

- Aggregate reports (no per-record filter, like the Maintenance one) simply omit the formula — don't set `SelectionFormulaConfigKey` on the button.
- Crystal syntax worth knowing: strings in single quotes (`'ABC'`), dates as `Date(2026,08,24)`, boolean operators spelled `and` / `or` — e.g. `{Command.StatusId} = 1 and {Command.Date} >= Date(2026,01,01)` .

5. Publish and wire it into the app

1. Upload the .rpt to `C:\AGILE_REPORTS\SRAMIL\` on the AgileInCloud server (ask support for access/the exact folder). Avoid accents and spaces in the file name — accented paths have caused encoding trouble before.

2. Add the keys under `CrystalReportsService` in appsettings.json (path + formula).

3. Drop the button on the page — it handles the "Generating..." state, errors (Snackbar) and the download; no extra page code needed:

```
<GerarRelatorioCrystalButton ReportPathConfigKey="MeuReportPath"
                             SelectionFormulaConfigKey="MeuReportFormula"
                             PlaceholderToken="{order_id}"
                             PlaceholderValue="@OrderId.ToString()"
                             CompanyDatabase="@CompanyDatabase"
                             DownloadFileNamePrefix="Relatorio_XYZ"
                             ButtonText="Relatório PDF" />
```

6. Troubleshooting

Symptom	Cause & fix
"Não foi possível comunicar com o serviço"	SOAP endpoint down or no network — check `BaseUrl` and open it in a browser (it should show the .asmx page).
SOAP fault: "Load Report Failed"	Wrong .rpt path or the file isn't on the server — verify the appsettings key character by character.
"Logon failed"	Invalid SQL credentials OR the company database isn't visible from the reports server.
Formula error	Test the formula in the Designer first (Report → Selection Formulas → Record) with a fixed value, before moving it to appsettings with a placeholder.
PDF renders but is empty	Usually the formula filters everything out (nonexistent Id) or the Command points at the wrong database — run the Command's SELECT by hand against the company database.
"Old" data in the PDF	The .rpt was saved with "Save Data with Report" enabled — resave without data and republish.

7. Visual guides — the QuestPDF reports

The reports the app generates itself follow the Ramil identity and are the model to replicate in Crystal: logo on the left over a green rule (`#1B6B3C`), KPI cards in light green (`#E8F2EC`), green table headers with white text, zebra rows (`#F4F7F5`) and a "Página X de Y" footer. Code in `Services/*/Export` and `Services/Reports/ReportTheme.cs` :

- **Mapa de Equipamento** — `EquipmentReportPdfBuilder` ("Relatório PDF" button on the equipment detail).
- **Relatório de Intervenções** — `InterventionReportPdfBuilder` (Tickets page).
- **Relatório de Manutenção** — `MaintenanceReportPdfBuilder` (Maintenance page).

Generate each one in the app, put the PDF next to the Designer, and replicate it section by section.